AI-Assisted Development Without Code

A Journey and Reference Guide

> How to build complex software with AI as your thinking partner — without needing to write a single line of code yourself.

By Bogs — documented in real time as the project was built

Preface

This document records the actual process used to design and build a complex AI orchestration stack from scratch. No code was written during this phase. Everything here happened before a single file was touched.

The goal is not to teach programming. It is to show that the hardest part of building software — thinking clearly about what you are building and why — does not require technical skill. It requires structured thinking, good questions, and a disciplined process.

This is that process, written down as it happened.

Part 1 — The Mindset

Chapter 1: Why Most People Use AI Wrong

The most common mistake: treating AI as a code generator.

You describe what you want. AI writes code. You paste it in. Something breaks. You ask AI to fix it. Something else breaks. An hour later you have code that technically runs but you do not understand, cannot maintain, and are afraid to change.

This is not AI-assisted development. This is AI-assisted guessing.

The problem is not the AI. The problem is the order of operations.

Code is the last thing you should produce, not the first. Before code exists there are decisions — hundreds of them. What does this thing actually do? How does it connect to everything else? What happens when it fails? Who is allowed to use it? These decisions do not write themselves. If you skip them and go straight to code, the code makes the decisions for you — badly, invisibly, and in ways that are expensive to change later.

AI is extraordinarily good at helping you make decisions — if you ask the right questions in the right order. It is a thinking partner, not a typing assistant.

Chapter 2: The Role of Structure

Structure is not bureaucracy. Structure is how you stop the same conversation from happening twice.

Without structure, every AI session starts from zero. You re-explain the project. AI re-suggests things you already rejected. You re-make decisions you made last week. Progress is slow not because AI is slow but because context is lost.

With structure, every session starts from where you left off. AI knows what has been decided. It knows what is still open. It knows what you are focused on right now. You spend the session thinking, not re-explaining.

The tools in this guide — Context Kit, Playbook, DDA — exist to provide that structure. They are not overhead. They are the reason the work compounds instead of resets.

Chapter 3: Thinking With AI vs Delegating To AI

There is a difference between:

“Build me an authentication system”

and:

“I need users to log in securely. We are using WorkOS. Auth should be enforced at the database boundary, not in application code. What does that flow look like?”

The first is delegation. You get back code you do not understand.

The second is thinking together. You get back a flow you can reason about, approve, and then ask AI to implement against a clear contract.

The discipline is asking questions before asking for output. Every decision made in conversation is a decision that does not get made badly inside the code.

Part 2 — The Process

Chapter 4: Start With Decisions, Not Code

The sequence that works:

```
1. What problem are we solving?        (PRD)
2. What does it consist of?            (structure)
3. How do the parts relate?            (flows and sequences)
4. What does each part do?             (DDA component docs)
5. How is it built?                    (code standards)
6. Build it                            (code)
7. Verify it                           (QA)
```

Most people start at step 6. By the time problems appear, steps 1–5 are implicit, inconsistent, and impossible to change without rewriting everything.

Starting at step 1 feels slow. It is not. The time spent on decisions is paid back immediately when implementation begins, because every question the code would have forced you to answer has already been answered.

Chapter 5: The Context Kit — Keeping AI on Track

AI has no memory between sessions. Every new chat starts from zero. The Context Kit solves this by externalising project memory into four files that are uploaded at the start of every session.

```
01-process.md    How we work + where we are right now
02-exploration.md  Open questions + closed decisions
03-execution.md  Guidelines + plan + tasks
04-progress.md   Append-only log of everything done
```

Two optional files for complex projects:

```
05-kanban.md   Ticketed work breakdown
06-qa.md       Test cases and sign-off
```

The discipline: update the files at the end of every session. Two minutes of filing prevents an hour of re-explanation next time.

Key rules learned in practice:

- Closed decisions stay closed unless explicitly reopened
- Progress log is append-only — never edit past entries
- Current State in process.md must always be specific, never vague
- Files are kept lean — archive when they grow unwieldy

Chapter 6: The Playbook Pipeline — From Idea to Code

The full pipeline:

```
Context Kit → PRD → DDA → Code → Rollout
```

Each stage has a clear input and a clear output. Nothing overlaps.

Context Kit captures thinking and decisions across sessions. It is the project management layer.

PRD (Product Requirements Document) defines what you are building and why. It is approved by the human before architecture begins. AI drafts it. Human approves it. No code touches something not in the PRD.

DDA (Document-Driven Architecture) defines how everything is structured. Every component has a document before it has code. The document is the contract. Code follows the contract, not the other way around.

Code is generated by AI against DDA contracts. If there is no DDA doc, there is no code. This is a hard rule, not a suggestion.

Rollout is a generated snapshot of the full architecture at a milestone. It serves as AI context for future sessions and as a human-readable release record.

Chapter 7: Flows and Sequences Before Components

One of the most important lessons from this project:

Map the flows before you define the components.

A component without a flow is a black box. You know what it does in isolation but not how it behaves in the system. Bugs live in the connections between components, not inside them.

The process:

1. Define the major runtime paths (what happens when X occurs)
2. Walk each path step by step, including failure paths
3. Find gaps, race conditions, and unresolved questions
4. Only then define the components that implement those paths

This feels like extra work. It prevents weeks of refactoring.

Chapter 8: The Foundation Takes Longer Than the Code

Expect to spend more time on structure than on implementation.

In this project:

- Multiple full sessions were spent on flows and sequences
- Architecture decisions were revisited and corrected multiple times
- The PRD went through review before a single DDA doc was written

This is not inefficiency. This is the work. The implementation, once the foundation is solid, goes quickly. The alternative — building on a shaky foundation — means the implementation goes quickly at first and then stops completely while you untangle decisions that should have been made at the start.

Part 3 — The Tools

Chapter 9: DDA — Documenting What You Are Building

DDA (Document-Driven Architecture) is the technical layer of the Playbook pipeline. Every component in the system has a structured document that defines:

- What it does (one paragraph, plain language)
- What it connects to

- What events it emits
- What concerns it depends on (logging, auth, error handling)
- Its current status (draft → active → deprecated)

Document types:

```
structure    Project or module manifest. Contains a living
             folder tree of everything in scope.

sequencer    A runtime flow coordinator. Owns sequence,
             not logic. Always includes a Flow section
             and a sequence diagram.

component    A single unit of logic. One responsibility.
             Max ~150 lines of code.

concern      A cross-cutting capability (logging, auth,
             error handling). Injected, never imported.

integration  A third-party service contract (WorkOS,
             Convex, llama.cpp).
```

Key rules:

- No code without a DDA doc
- No DDA doc without an approved flow
- Components are small — god-components are forbidden
- Files are small — max ~150 lines
- One responsibility per component, per file, per doc

Chapter 10: How the Tools Work Together

```
Context Kit keeps sessions coherent.
PRD defines what to build.
DDA defines how to build it.
Flows connect PRD intent to DDA structure.
Code implements DDA contracts.
QA verifies against PRD criteria.
```

None of these replaces the others. Each has a distinct job. The discipline is using them in order and not skipping steps.

Part 4 — The Journey

Session 1 — Starting Point

Date: 2026-05-10

Where we started: An existing orchestrator project with raket-screening and trading as connected projects. The orchestrator was already partially built but tightly coupled to raket-screening logic. A refactor was needed.

First decision: Before touching any code, define the full architecture of what this should become. This meant resisting the urge to fix the immediate problem and instead asking: what is the right design?

Key insight of this session: The question "what should we do next?" was replaced with "what decisions do we need to make first?" That shift changed everything about how the session proceeded.

What we did:

- Reviewed existing kit files and PRD
- Identified that the orchestrator needed to be fully decoupled from project-specific logic
- Began defining the monorepo structure
- Established that DDA document type for flow coordinators would be called `sequencer`
- Defined agent role catalog (builder, researcher, qa, checker, vision, assistant)
- Decided on Convex (self-hosted, Postgres-backed) as primary DB
- Decided on Docker Compose for process orchestration
- Mapped the boot sequence across 6 phases
- Began Flow 1 (task submission → Ralph loop → result)

Decisions made:

- Orchestrator app keeps its name; DDA type renamed to sequencer
- Ralph loop fixed at 3 attempts
- Role selection is pure TypeScript — no model needed
- 3-tier streaming: agent-runner buffers, Models API debounces, Convex writes sparingly

- vision role is orchestrator-level, not raket-screening specific
- discovery, models-api run as independent apps
- WorkOS Free tier for auth, JWT at Convex boundary
- Design system as standalone package (Shadcn + Tailwind dark theme)
- All actions audited throughout entire stack

What we learned:

- Naming matters more than it seems. Ambiguous names (orchestrator meaning four different things) cause confusion that compounds. Rename early.
- The temptation to jump to code is strong. Resist it every time. Every decision made in conversation is a decision not made badly in code.
- Flows before components. Always.
- Resource constraints (limited machine) must be first-class architectural concerns, not afterthoughts.

Files produced this session:

- PRD.md (approved)
- 01-process.md through 06-qa.md (updated)
- Flow 0 (boot sequence) — defined
- Flow 1 (task lifecycle) — defined and reviewed
- Flow 2 (project connection lifecycle) — defined

Still open:

- Flows 3–10 to be defined
- DDA scaffold not yet started
- agent-executor.md DDA doc needs updating before code changes

Part 5 — Replication Guide

How to Start Your Own Project This Way

Step 1: Resist the urge to build anything

The first session should produce no code. It should produce clarity.

Start by describing the project in two sentences. Then ask AI: “What decisions do we need to make before we can design this?”

Work through those decisions. Document them in your kit files.

Step 2: Set up your Context Kit

Copy the four template files. Fill in:

- What phase you are in
- What you are focused on right now
- Two or three key guidelines
- One progress entry with today’s date

This takes ten minutes. It pays back in every future session.

Step 3: Write the PRD with AI

Once decisions are made, ask AI to draft a PRD from your closed decisions. Review it carefully. Every line should reflect something you actually decided. Approve it only when it is accurate.

The PRD is the contract between you and the project. Do not rush it.

Step 4: Map the flows

Before any component is defined, walk through the major runtime paths:

- What happens when a user does X?
- What happens when something fails?
- What happens when the system restarts?

Walk each path step by step with AI. Write down every step. Find the gaps and fix them in conversation, not in code.

Step 5: Define the structure

With flows approved, define the structure:

- What apps exist?
- What packages are shared?
- What are the boundaries between them?
- What are the dependencies?

Draw the folder tree. Name everything consistently. Resolve ambiguity before it becomes a bug.

Step 6: Write DDA docs top-down

Start with the project `_index.md`. Then module `_index.md` files. Then sequencer docs (flows). Then component docs.

Never write a component doc before its parent sequencer doc exists. Never write code before the component doc exists.

Step 7: Build

With DDA docs approved, ask AI to implement each component against its doc. Review the output. Run the checks. If code diverges from the doc, update the doc first, then the code.

Common Mistakes to Avoid

Skipping the PRD. Without a PRD, scope is undefined. Everything feels like it belongs in the project. Nothing is ever done.

Writing component docs before flows. A component doc without a flow is a guess. The flow tells you what the component actually needs to do.

Letting the AI decide the architecture. AI is a good advisor. It is a poor architect when given free rein. Ask it to propose. You decide. Approve explicitly.

Ignoring naming. If the same word means different things in different contexts, bugs follow. Name things precisely and consistently from the start.

Building before the foundation is solid. Momentum feels like progress. Rework is expensive. One hour of design prevents four hours of refactoring.

Closing Note

This guide will grow as the project progresses. Each session adds a chapter to Part 4. Each lesson learned updates Part 5.

The goal is not a perfect document. It is an honest record of how a complex project was built, decision by decision, with AI as a thinking partner rather than a typing service.

If you can read this and replicate the process, it has done its job.